

程序设计 II 第 7 周 题目汇总（完整版）

来源：Matrix·课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。选择题含标准答案。

共 7 项

25 程设II-周 7-课后 1

类型：编程题

题面

const 与 mutable 结合用法

实现 Test 类，要求：

1. **成员变量：**
 - int m_value：普通成员
 - mutable int m_count：可变成员，记录 show() 被调用的次数
2. **构造函数：**
 - Test(int val = 0)：初始化 m_value = val，m_count = 0，输出 "Constructor: val"
3. **拷贝构造函数：**
 - 拷贝 m_value，但 m_count 重置为 0
 - 输出 "Copy Constructor: val"
4. **析构函数：**
 - 输出 "Destructor: val"
5. **成员函数：**
 - void modify(int val)：修改 m_value = val，输出 "Modify: val"
 - void show() const：m_count++，输出 "Show: value=val, count=m_count"

输入数据

随机正整数

输出数据

程序输出构造函数、拷贝构造函数和析构函数的调用信息。

输入示例 1

1

输出示例 1

Constructor: 10
Copy Constructor: 10
Modify: 20
Copy Constructor: 20
Show: value=20, count=1
Destructor: 20
Destructor: 20
Destructor: 10

提交文件：solution.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "solution.hpp"
using namespace std;
```

```
Test func1(Test t) {
    return t;
}
```

```
void func2(Test t) {
    t.show();
}
```

```
int main() {
    int rank;
    cin >> rank;
    rank %= 2;

    const Test a(10);

    if (rank == 0) {
        a.show();
        a.show();
    } else {
        Test b = a;
        b.modify(20);
        func2(b);
    }

    return 0;
}
```

文件 solution.hpp：

```
#ifndef SOLUTION_HPP
#define SOLUTION_HPP

#include <iostream>

class Test {
private:
    int m_value;
    mutable int m_count;

public:
    Test(int val = 0);
    Test(const Test& other);
    ~Test();

    void modify(int val);
    void show() const;
};

#endif
```

25 程设II-周 7-课后 2

类型：编程题

题面

整数数组管理器

题目描述

实现 IntArray 类，管理一个动态整数数组。

成员变量

- int* m_data：指向动态数组的指针
- int m_size：当前元素个数
- int m_capacity：数组容量
- mutable int m_accessCount：记录 get() 被调用的次数

要求

1. 拷贝构造函数 IntArray(const IntArray& other)

- 实现深拷贝
- m_accessCount 重置为 0
- 输出 "Copy Constructor"

2. 析构函数 ~IntArray()

- 释放资源
- 输出 "Destructor"

3. 成员函数

- void add(int val)：添加元素，容量不足时扩容（翻倍），输出 "Add: val"
- int get(int index) const：获取元素，m_accessCount++，输出 "Get: val"，越界返回 -1

输入数据

随机正整数

输出数据

程序输出构造函数、拷贝构造函数和析构函数的调用信息。

输入示例 1

1

输出示例 1

Constructor
Add: 10
Add: 20
Copy Constructor
Add: 30
arr1: 10 20
arr2: 10 20 30
Destructor
Destructor

提交文件：solution.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "solution.hpp"
using namespace std;

void func1(IntArray arr) {
    arr.get(0);
}

IntArray func2() {
    IntArray arr(3);
    arr.add(100);
    return arr;
}
```

```

}

int main() {
    int choice;
    cin >> choice;
    choice %= 3;

    if (choice == 0) {
        const IntArray arr(3);
        arr.get(0);
        arr.get(0);
        cout << "Access count: " << arr.getAccessCount() << endl;
    }
    else if (choice == 1) {
        IntArray arr1(2);
        arr1.add(10);
        arr1.add(20);

        IntArray arr2 = arr1;
        arr2.add(30);

        cout << "arr1: ";
        arr1.printAll();
        cout << "arr2: ";
        arr2.printAll();
    }
    else {
        IntArray arr1(2);
        arr1.add(5);

        IntArray arr2 = func2();
        func1(arr1);

        arr2.get(0);
    }

    return 0;
}

```

文件 solution.hpp :

```

#ifndef SOLUTION_HPP
#define SOLUTION_HPP

#include <iostream>
using namespace std;

class IntArray {
private:
    int* m_data;

```

```

int m_size;
int m_capacity;
mutable int m_accessCount;

public:
IntArray(int capacity) : m_size(0), m_capacity(capacity), m_accessCount(0) {
    m_data = new int[m_capacity];
    cout << "Constructor" << endl;
};

IntArray(const IntArray& other);
~IntArray();

void add(int val);
int get(int index) const;

void printAll() const {
    for (int i = 0; i < m_size; i++) {
        cout << m_data[i] << " ";
    }
    cout << endl;
};

int getSize() const {
    return m_size;
};

int getAccessCount() const {
    return m_accessCount;
};
};

#endif

```

25 程设II-周 7-课堂 1

类型：编程题

题面

实现 Student 类的拷贝构造函数

Student 类包含：

- char name[20]
- int age

- **double** score

要求实现：

- 构造函数 Student(const char* n, int a, double s)
- 拷贝构造函数 Student(const Student& s)
- 成员函数 change()
- 成员函数 show()

其中：

- change() 将年龄加 1，得分加 5(满分 100 分)
- show() 按格式输出：名字 年龄 成绩

输入数据

输入三行：第一行为字符串表示名字；第二行为整数表示年龄；第三行为浮点数表示得分。

输出数据

三行（名字 年龄 得分）

输入示例 1

```
tom
6
99
```

输出示例 1

```
tom 6 99
tom 6 99
tom 7 100
```

输入示例 2

```
jerry
6
90
```

输出示例 2

```
jerry 6 90
jerry 6 90
jerry 7 95
```

提交文件：solution.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "solution.hpp"
```

```

using namespace std;

Student func(Student s) {
    s.change();
    return s;
}

int main() {
    string name;
    int age;
    double score;

    cin >> name;
    cin >> age;
    cin >> score;

    Student s1(name.c_str(), age, score);
    Student s2 = s1;
    Student s3 = func(s2);

    s1.show();
    cout<<endl;
    s2.show();
    cout<<endl;
    s3.show();

    return 0;
}

```

文件 solution.hpp :

```

#ifndef SOLUTION_HPP
#define SOLUTION_HPP

#include <iostream>
#include <cstring>
using namespace std;

class Student {
public:
    Student(const char* n, int a, double s);
    Student(const Student& s);
    void change();
    void show();

private:
    char name[20];
    int age;
    double score;
};

```


#endif

25 程设II-周 7-课堂 2

类型：编程题

题面

实现 MyString 类

实现 MyString 类，类中使用 string 存储字符串。

需要实现：

1. 构造函数 MyString(const string& str)
2. 拷贝构造函数 MyString(const MyString& s)
3. 成员函数 set(const string& str)
4. 成员函数 show()

程序功能：

- 输入两个字符串
- s1 用第一个字符串初始化
- s2 由 s1 拷贝构造
- 输出 s2
- 修改 s2
- 输出 s1 和 s2

输入数据

两行字符串

输出数据

三行输出，分别是 s2、s1、修改后的 s2

输入示例 1

hello
world

输出示例 1

hello
hello
world

提交文件：solution.cpp

平台提供的初始文件

文件 main.cpp :

```
#include <iostream>
#include "solution.hpp"
using namespace std;

int main() {
    string str1, str2;
    cin >> str1;
    cin >> str2;

    MyString s1(str1);
    MyString s2 = s1;

    s2.show();
    cout<<endl;
    s2.set(str2);

    s1.show();
    cout<<endl;;
    s2.show();

    return 0;
}
```

文件 solution.hpp :

```
#ifndef SOLUTION_HPP
#define SOLUTION_HPP

#include <iostream>
#include <string>
using namespace std;

class MyString {
public:
    MyString(const string& str);
    MyString(const MyString& s);
    void set(const string& str);
    void show();

private:
    string data;
};

#endif
```

25 程设II-周 7-课堂 3

类型：编程题

题面

观察拷贝构造函数调用次数

实现 Test 类，要求：

- 默认构造函数输出 constructor
- 拷贝构造函数输出 copy constructor
- 析构函数输出 destructor

已给出 main.cpp，程序会随机执行两段代码中的一段。请根据程序运行结果，理解拷贝构造函数的调用时机。

输入数据

随机正整数

输出数据

程序输出构造函数、拷贝构造函数和析构函数的调用信息。

输入示例 1

0

输出示例 1

```
constructor
copy constructor
copy constructor
destructor
destructor
destructor
```

提交文件：solution.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "solution.hpp"
using namespace std;
```

```
Test func1(Test t) {
    return t;
}
```

```

void func2(Test t) {
}

int main() {
    int rank;
    cin>>rank;
    rank %= 2;

    Test a;

    if (rank == 0) {
        Test b = a;
        func2(b);
    } else {
        Test b = a;
        Test c = func1(b);
    }

    return 0;
}

```

文件 solution.hpp :

```

#ifndef SOLUTION_HPP
#define SOLUTION_HPP

#include <iostream>
using namespace std;

class Test {
public:
    Test();
    Test(const Test& t);
    ~Test();
};

#endif

```

25 程设II-周 7-课堂 4

类型：编程题

题面

序列副本

设计一个 Seq 类，用来管理一个动态整数序列，要求如下：

类的要求：

1. 包含三个数据成员
 - n：表示序列长度，类型为 int
 - data：指向动态整数数组的指针，类型为 int*
 - total：静态数据成员，表示当前存在的 Seq 对象个数，类型为 static int
2. 提供构造函数 Seq(int n = 0)
 - 使用参数 n 创建长度为 n 的动态数组
 - 当 n > 0 时申请动态空间
 - 对象创建成功后，total+1
3. 提供拷贝构造函数 Seq(const Seq& other)
 - 完成深拷贝
 - 新对象创建成功后，total+1
4. 提供析构函数
 - 释放动态数组空间
 - 对象销毁时，total-1
5. 提供成员函数 void input()
 - 读入当前序列的 n 个整数
6. 提供成员函数 void set(int index, int value)
 - 将下标为 index 的元素修改为 value；下标从 0 开始
7. 提供成员函数 int longestNonDec() const
 - 返回该序列中**最长连续不下降子段**的长度，“不下降”指后一项 $\geq$ 前一项
 - 例：序列 1 2 2 1 3 的**最长连续不下降子段**长度为 3，因为 1 2 2 满足要求
8. 提供成员函数 int diffCount(const Seq& other) const
 - 比较当前对象与参数对象对应位置上的元素，返回数值不同的位置个数
 - 注：本题中参与比较的两个序列长度相同
9. 提供成员函数 static int getCount()
 - 返回当前存在的 Seq 对象个数

要求：

- 不使用 vector、string 等现成容器代替动态数组
- 拷贝构造函数必须实现深拷贝
- longestNonDec()和 diffCount()按题意体现 const 的使用

输入数据

第一行：一个整数 n，表示序列长度 第二行：n 个整数，表示序列元素，以空格间隔 第三行：两个整数 index 和 value，表示将原序列下标为 index 的元素改为 value

注：

- $1 \leq n \leq 100$
- 下标 index 合法
- value 为普通 int 范围内整数

输出数据

第一行输出：修改后的原序列的**最长连续不下降子段长度** 第二行输出：拷贝得到的副本序列的**最长连续不下降子段长度** 第三行输出：两个序列中对应位置不同的**元素个数**，以及当前存在**对象个数**，中间以空格分隔

输入示例 1

```
5
1 2 2 1 3
0 9
```

输出示例 1

```
2
3
1 2
```

输入示例 2

```
6
3 1 2 2 5 4
1 6
```

输出示例 2

```
3
4
1 2
```

输入示例 3

```
1
7
0 3
```

输出示例 3

```
1
1
1 2
```

提交文件：Seq.hpp、Seq.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "Seq.hpp"
```

```

using namespace std;

int main() {
    int n;
    cin >> n;

    Seq s1(n);
    s1.input();

    Seq s2 = s1;

    int index, value;
    cin >> index >> value;
    s1.set(index, value);

    const Seq& ref = s2;

    cout << s1.longestNonDec() << endl;
    cout << ref.longestNonDec() << endl;
    cout << s1.diffCount(ref) << " " << Seq::getCount() << endl;

    return 0;
}

```

25 程设II-周 7-理论题

类型：选择题（20 题）

第 1 题

关于 C++ 中对象的存储与释放，下列说法正确的是

- A. 用 new 创建的对象通常位于栈区，离开作用域后会自动释放
- B. 局部对象通常位于堆区，必须使用 delete 才能释放
- C. 用 new 创建的对象需要显式释放，否则可能造成内存泄漏
- D. 对指针执行 delete 后，指针会自动变为 nullptr

答案：C

第 2 题

有关下列代码说法正确的是

```

#include <iostream>
using namespace std;

```

```

int main() {
    int* p = new int(5);
}

```

```
int* q = p;  
delete p;  
return 0;  
}
```

- A. q 会自动变为 nullptr
- B. 此时 q 仍然指向原地址值，但该地址已经失效
- C. 此时仍可通过*q 安全访问原来的值 5
- D. 此时再次执行 delete q 一定是安全的

答案：B

第 3 题

关于构造函数的成员初始化列表，下列说法正确的是

- A. 成员初始化列表中的初始化发生在构造函数体执行之后
- B. 在构造函数体内对成员赋值，与成员初始化列表初始化完全等价
- C. 成员初始化列表中的初始化发生在进入构造函数体之前
- D. 所有成员都必须在构造函数体内赋值，不能在初始化列表中处理

答案：C

第 4 题

设有如下定义：

```
int a = 10, b = 20;  
const int* p1 = &a;  
int* const p2 = &a;
```

下列说法正确的是

- A. p1 和 p2 都不能修改所指向的值
- B. p1 不能通过它修改所指向的值，但可以改为指向别处；p2 可以通过它修改所指向的值，但不能改为指向别处
- C. p1 不能改为指向别处，但可以通过它修改所指向的值；p2 正好相反
- D. p1 和 p2 都既不能修改指向，也不能修改所指向的值

答案：B

第 5 题

关于普通类对象在内存中的组成，下列说法正确的是

- A. 对象中包含该类所有成员函数的代码副本
- B. 类的静态数据成员属于每个对象各自独立拥有的一份数据
- C. 普通对象通常包含其非静态数据成员所需的存储空间
- D. 不同对象之间会共享各自的非静态数据成员

答案：C

第 6 题

下列代码的输出结果是

```
#include <iostream>
using namespace std;

class Test {
public:
    Test() { cout << "C "; }
    ~Test() { cout << "D "; }
};

int main() {
    Test t1;
    Test* p = new Test;
    delete p;
    return 0;
}
```

- A. C D C D
- B. C C D D
- C. C C C D
- D. C D D C

答案：B

第 7 题

关于拷贝构造函数，下列说法正确的是

- A. 对象已经存在后，再执行赋值语句，一定会调用拷贝构造函数
- B. 用一个已有对象去初始化同类新对象时，通常会调用拷贝构造函数
- C. 拷贝构造函数的参数必须按值传递
- D. 拷贝构造函数只能手动调用，编译器不会自动调用

答案：B

第 8 题

有关下列代码的说法正确的是：

```
#include <iostream>
using namespace std;

class A {
private:
    const int x;
```

```
public:
    A() {
        x = 10;
    }
};
```

- A. 代码可以正常编译，因为 const 成员可以先定义再在构造函数体内赋值
- B. 代码不能通过编译，因为 const 数据成员必须在成员初始化列表中初始化
- C. 代码不能通过编译，因为类中不能定义 const 数据成员
- D. 代码可以正常编译，但运行时会报 error

答案：B

第 9 题

有关下列代码，哪一项说法正确

```
#include <iostream>
using namespace std;

class A {
private:
    int x;
public:
    A(int v) : x(v) {}
    void f() { cout << x << endl; }
    void g() const { cout << x << endl; }
};

int main() {
    const A a(10);
    a.g();
    a.f();
    return 0;
}
```

- A. 代码可以正常编译并输出两次 10
- B. 代码不能通过编译，因为 const 对象不能调用任何成员函数
- C. 代码不能通过编译，因为 const 对象不能调用非 const 成员函数 f()
- D. 代码不能通过编译，因为 const 成员函数 g() 不能访问数据成员 x

答案：C

第 10 题

下面程序的输出结果是

```
#include <iostream>
using namespace std;
```

```

class Counter {
private:
    static int cnt;
public:
    Counter() { cnt++; }
    void show() const { cout << cnt << " "; }
};

```

```

int Counter::cnt = 0;

```

```

int main() {
    Counter a;
    Counter b;
    a.show();
    b.show();
    return 0;
}

```

- A. 1 1
- B. 1 2
- C. 2 2
- D. 编译错误

答案：c

第 11 题

下面程序运行过程中拷贝构造函数一共会被调用几次

```

#include <iostream>
using namespace std;

```

```

class A {
public:
    A() {}
    A(const A& other) {
        cout << "copy ";
    }
};

```

```

void fun(A x) {
    A y = x;
}

```

```

int main() {
    A a;
    fun(a);
    return 0;
}

```

- A. 0
- B. 1
- C. 2
- D. 3

答案：C

第 12 题

有关下列代码说法正确的是

```
#include <iostream>
using namespace std;
```

```
class A {
public:
    A(int x) {}
};
```

```
int main() {
    A* p = new A[3];
    delete[] p;
    return 0;
}
```

- A. 代码可以正常编译，因为 new A[3]会自动调用 A(int)三次
- B. 代码不能通过编译，因为创建对象数组时需要能够调用默认构造函数
- C. 代码不能通过编译，因为对象数组不能用 delete[]释放
- D. 代码可以正常编译，但 delete[] p 一定会运行错误

答案：B

第 13 题

关于 C++ 中的 =delete 和 =default，下列说法正确的是

- A. =delete 表示该函数由编译器自动生成默认实现
- B. =default 表示该函数不可被调用
- C. =delete 可用于显式禁止某个函数被调用，=default 可用于要求编译器生成默认版本
- D. =delete 和 =default 只能用于析构函数

答案：C

第 14 题

下列代码最可能出现的问题是：

```
#include <iostream>
#include <cstring>
```

```

using namespace std;

class String {
private:
    char* data;
public:
    String(const char* s) {
        data = new char[strlen(s) + 1];
        strcpy(data, s);
    }

    ~String() {
        delete[] data;
    }
};

int main() {
    String s1("hello");
    String s2 = s1;
    return 0;
}

```

- A. 程序一定无法通过编译，因为类中含有指针成员
- B. 程序运行完全安全，因为系统会自动完成深拷贝
- C. 程序可能因浅拷贝导致多个对象指向同一块内存，析构时发生重复释放
- D. 程序中 `s2 = s1` 会调用赋值运算符，而不是拷贝构造函数

答案：C

第 15 题

以下程序的输出结果是

```

#include <iostream>
using namespace std;

```

```

class A {
private:
    int x;
public:
    A(int v) : x(v) {}

    int get() const {
        return x;
    }
};

void show(const A& a) {
    cout << a.get() << endl;
}

```

```
int main() {  
    A a(10);  
    show(a);  
    return 0;  
}
```

- A. 0
- B. 10
- C. 编译错误，因为 const A&不能绑定到普通对象
- D. 编译错误，因为 show 中不能调用 const 成员函数

答案：B

第 16 题

以下程序的输出结果是

```
#include <iostream>  
using namespace std;
```

```
class X {  
public:  
    X(int v) {  
        cout << v << " ";  
    }  
};
```

```
class A {  
private:  
    X x;  
    X y;  
public:  
    A() : y(2), x(1) {}  
};
```

```
int main() {  
    A a;  
    return 0;  
}
```

- A. 1 2
- B. 2 1
- C. 1
- D. 编译错误

答案：A

第 17 题

以下程序的输出结果是

```
#include <iostream>
using namespace std;

class A {
public:
    A() {}
    A(const A&) {
        cout << "copy ";
    }
    A& operator=(const A&) {
        cout << "assign ";
        return *this;
    }
};

int main() {
    A a;
    A b = a;
    A c;
    c = a;
    return 0;
}
```

- A. copy assign
- B. assign copy
- C. copy copy
- D. assign assign

答案：A

第 18 题

有关以下代码说法正确的是：

```
#include <iostream>
using namespace std;

class A {
public:
    A() = delete;
    A(int x) {}
};

int main() {
    A a;
    A b(10);
}
```

```
    return 0;
}
```

- A. 代码可以正常编译，因为 A b(10)合法
- B. 代码不能通过编译，因为类中只要出现=delete 就会整体失效
- C. 代码不能通过编译，因为 A a;试图调用已删除的默认构造函数
- D. 代码不能通过编译，因为 A(int)不能与 A() = delete 同时存在

答案：C

第 19 题

以下程序的输出结果是

```
#include <iostream>
using namespace std;
```

```
class A {
private:
    int x;
    static int y;
public:
    A(int v) : x(v) {}
    void setX(int v) { x = v; }
    static void setY(int v) { y = v; }
    void show() const { cout << x << " " << y << endl; }
};
```

```
int A::y = 0;
```

```
int main() {
    A a(1), b(2);
    A::setY(5);
    a.show();
    b.show();
    return 0;
}
```

- A. 1 0 / 2 0
- B. 1 5 / 2 5
- C. 5 1 / 5 2
- D. 1 5 / 1 5

答案：B

第 20 题

阅读下列代码。若希望该类在“用一个对象初始化另一个对象”时也能正确工作，最应补充或修改的是哪一项？


```
#include <iostream>
#include <cstring>
using namespace std;

class String {
private:
    char* data;
public:
    String(const char* s) {
        data = new char[strlen(s) + 1];
        strcpy(data, s);
    }

    ~String() {
        delete[] data;
    }
};

int main() {
    String s1("hello");
    String s2 = s1;
    return 0;
}
```

- A. 把析构函数删除即可
- B. 把 char* data 改成 int data
- C. 补充拷贝构造函数，在其中重新申请空间并复制字符串内容
- D. 把构造函数参数改成 char* s，去掉 const

答案：c
